AI для .NET-розробника

Том 2 · Промпти й GitHub Copilot

Розділи 4–6

Розділ 4. Промпт-інжиніринг для коду

4.1. Анатомія хорошого промпту

Добрий запит складається з п'яти частин. Не всі обов'язкові, але чим більше — тим краще результат.

Частина	Питання	Приклад
Роль	Хто ти?	«Ти досвідчений .NET-розробник»
Задача	Що зробити?	«Додай пагінацію в ендпоінт GET /todos»
Контекст	Що вже є?	«.NET 9, Minimal API, EF Core, SQLite»
Обмеження	Чого уникати?	«Без нових NuGet-пакетів; не змінюй публічні контракти»
Формат	Як відповісти?	«Покажи лише змінені методи та короткий список змін»

4.2. Поганий і добрий промпт

```
Погано:  зроби щоб працювало

Добре:   У TodoService.GetAllAsync() метод повертає всі записи.
         Додай параметри page і pageSize (за замовчуванням 1 і 20, максимум 100),
         використай Skip/Take у EF Core, поверни PagedResult<TodoDto>.
         Додай юніт-тест xUnit на межові значення. Стиль — як у наявному коді.
```

4.3. Техніки

Техніка	Суть	Коли
Конкретика	Версії, назви класів, очікуваний результат	Завжди
Few-shot	Дати 1–2 приклади потрібного стилю	Нетиповий формат, іменування
Крок за кроком	Спершу попросити план, потім реалізацію	Задачі на кілька файлів
Ітерація	Уточнювати: «зроби асинхронно», «прибери дублювання»	Майже завжди
Критика	«Знайди в цьому коді 5 проблем» / «Які крайні випадки не враховано?»	Рев'ю
Дозвіл не знати	«Якщо не впевнений — скажи, не вигадуй API»	Нові бібліотеки
Поділ задачі	Одна відповідальність на запит	Великі фічі

4.4. Шаблони для щоденних задач .NET

```
Пояснення:  Поясни цей метод по кроках. Вкажи потенційні баги та складність.
Тести:      Напиши xUnit-тести для класу X з Moq. Покрий щасливий шлях, null, порожні колекції, винятки.
Рефакторинг: Розбий цей метод на менші, не змінюючи поведінки. Поведінку підтверди існуючими тестами.
Баг:        Є виняток: [стек-трейс]. Код: [фрагмент]. Назви 3 найімовірніші причини й перевірки для кожної.
Міграція:   Перепиши цей код із синхронного на async/await, збережи сигнатури публічного API.
LINQ/SQL:   Перетвори SQL-запит у LINQ для EF Core; поясни, який SQL буде згенеровано.
```

4.5. Антипатерни

- **Занадто великий запит:** «зроби весь застосунок» — результат поверхневий.
- **Відсутність версій:** без «.NET 9» отримаєте API старого фреймворку.
- **Сліпе копіювання:** без збірки й тестів.
- **Нескінченний чат:** після 30+ повідомлень якість падає — почніть нову розмову з підсумком.
- **Приховане очікування:** якщо щось важливо (стиль, бібліотека) — скажіть прямо.

> **Завдання 4.** Візьміть погано сформульований запит «додай логування» і перепишіть за схемою 4.1 для вашого TodoApi (ILogger, рівні, що логувати, чого не логувати). Порівняйте відповіді на обидва варіанти.

Розділ 5. GitHub Copilot для .NET

5.1. Де працює Copilot

Середовище	Особливості
Visual Studio 2022 / 2026	Copilot Chat у бічній панелі, автодоповнення, Agent mode, контекстні дії у меню (Explain, Fix, Generate Tests), допомога в налагодженні та в повідомленні коміту
VS Code + C# Dev Kit	Повний набір: Ask / Edit / Agent, prompt files, custom agents, MCP
JetBrains Rider	Плагін GitHub Copilot: чат і автодоповнення
Copilot CLI	Агент у терміналі (розділ 11)
github.com	Чат, рев'ю PR, cloud agent (розділ 12)

Доступність функцій відрізняється між середовищами й версіями; нові можливості зазвичай першими з'являються у VS Code.

5.2. Три основні взаємодії

Автодоповнення (inline suggestions)

Сірий текст-підказка під час набору. `Tab` — прийняти, `Esc` — відхилити. Найкраще працює, коли ви пишете змістовну назву методу або коментар-опис.

```
// Повертає true, якщо рядок є валідним email за RFC-спрощеним правилом
public static bool IsValidEmail(string? value)
{   // ← Copilot запропонує тіло
```

Next Edit Suggestions

Підказує наступну правку в іншому місці коду після вашої зміни (наприклад, оновити виклики після перейменування параметра), якщо функція увімкнена у вашому середовищі.

Чат

Діалог із можливістю додати контекст. Корисні slash-команди: `/explain`, `/fix`, `/tests`, `/doc`.

5.3. Режими чату

Режим	Що робить	Коли
Ask	Лише відповідає, файли не змінює	Питання, навчання, огляд коду
Edit	Правки у вибраних вами файлах із показом diff	Точкові зміни, коли ви знаєте місце
Agent	Сам шукає файли, редагує, запускає команди, виправляє помилки збірки/тестів	Фічі на кілька файлів
Plan (де доступний)	Складає план без змін коду	Перед великими задачами

> **Безпека агента:** спершу залишайте підтвердження дій увімкненими. Перед «Keep/Accept» дивіться diff. Не вмикайте автозатвердження в проєктах із секретами чи продакшн-доступом.

5.4. Типовий робочий процес з Agent

1. Створіть гілку: `git switch -c feature/todo-pagination` — щоб легко відкотити.
2. У режимі Plan або Ask попросіть план.
3. Перейдіть в Agent: «Реалізуй план, потім виконай `dotnet build` і `dotnet test` і виправ помилки».
4. Перегляньте diff по файлах.
5. Запустіть застосунок і перевірте вручну.
6. Закомітьте з осмисленим повідомленням.

5.5. Вибір моделі

У випадаючому списку чату можна змінити модель. Швидкі — для дрібниць, потужніші — для складного. Режим Auto обирає сам. Різні моделі споживають різну кількість premium requests згідно з вашим тарифом — актуальні множники дивіться в документації GitHub.

5.6. Типові помилки Copilot у .NET

Проблема	Як виявити/виправити
Вигаданий метод або перевантаження	Помилка компіляції; попросіть дати посилання на документацію
Застарілий API (наприклад, старий стиль Startup.cs)	Явно вказувати версію .NET і «Minimal API / top-level statements»
Пропущений `await` / `CancellationToken`	Рев'ю й аналізатори Roslyn
Нечесні тести, що нічого не перевіряють	Дивіться Assert; ламайте код навмисно і перевіряйте, що тест падає
Секрети в коді	Інструкція проєкту (розділ 7) + secret scanning

> **Завдання 5.** У гілці виконайте повний цикл 5.4: попросіть Agent додати пагінацію й тест. Зафіксуйте: що агент зробив правильно, що довелося виправити вручну.

Розділ 6. Керування контекстом

6.1. Чому контекст — головне

Модель бачить лише те, що ви їй дали. «Погана» відповідь найчастіше означає «нестача або сміття в контексті». Контекстне вікно обмежене, тому треба давати **потрібне й достатнє**.

6.2. Способи додати контекст

Спосіб	Що додає
Виділений код	Конкретний фрагмент
Відкритий файл	Активний файл часто додається автоматично
`#file` / «Add context» / `#`-посилання	Вказаний файл чи папка
`#codebase` / `@workspace`	Пошук по всьому проєкту; Copilot сам підбирає релевантні файли
`#selection`, `#terminalLastCommand`, `#problems` (VS Code)	Виділення, вивід терміналу, помилки з панелі Problems
Зображення	Скріншот помилки чи макету UI
Веб-посилання / `#fetch`	Вміст сторінки документації (де підтримується)
Файли інструкцій	Постійний контекст проєкту (розділи 7–9)

Точні назви змінних можуть відрізнятися між Visual Studio та VS Code — перевіряйте підказки після введення `#`.

6.3. Гігієна контексту

- **Менше — краще:** 3 потрібні файли краще за 30 випадкових.
- **Нова задача — новий чат.** Старий контекст заважає.
- **Довгий чат:** попросіть «зроби підсумок домовленостей» і почніть новий чат із цього підсумку.
- **Дайте вихідні дані:** повний стек-трейс, а не «воно падає».
- **Виключайте шум:** `bin/`, `obj/`, згенерований код, великі JSON.
- **Не додавайте секрети** (appsettings із паролями, .env).

6.4. Приклад: налагодження з контекстом

```
Контекст: #file:OrderService.cs #file:OrderServiceTests.cs
Помилка тесту (вивід): [вставити]
Задача: поясни причину падіння; запропонуй мінімальну правку в OrderService,
не змінюючи тести. Якщо потрібна додаткова інформація — запитай.
```

6.5. Пам'ять і контекст між сесіями

Чат **не пам'ятає** попередні розмови. Усе, що має діяти постійно (стиль, команди збірки, архітектурні правила), записуйте у файли інструкцій у репозиторії. Це тема наступного тому.

> **Завдання 6.** Спровокуйте нестачу контексту: попросіть «виправ баг у сервісі» без файлів. Потім додайте файл, тест і стек-трейс. Опишіть різницю у відповідях.

> **Далі — Том 3:** файли інструкцій, prompt files, skills, custom agents, hooks.